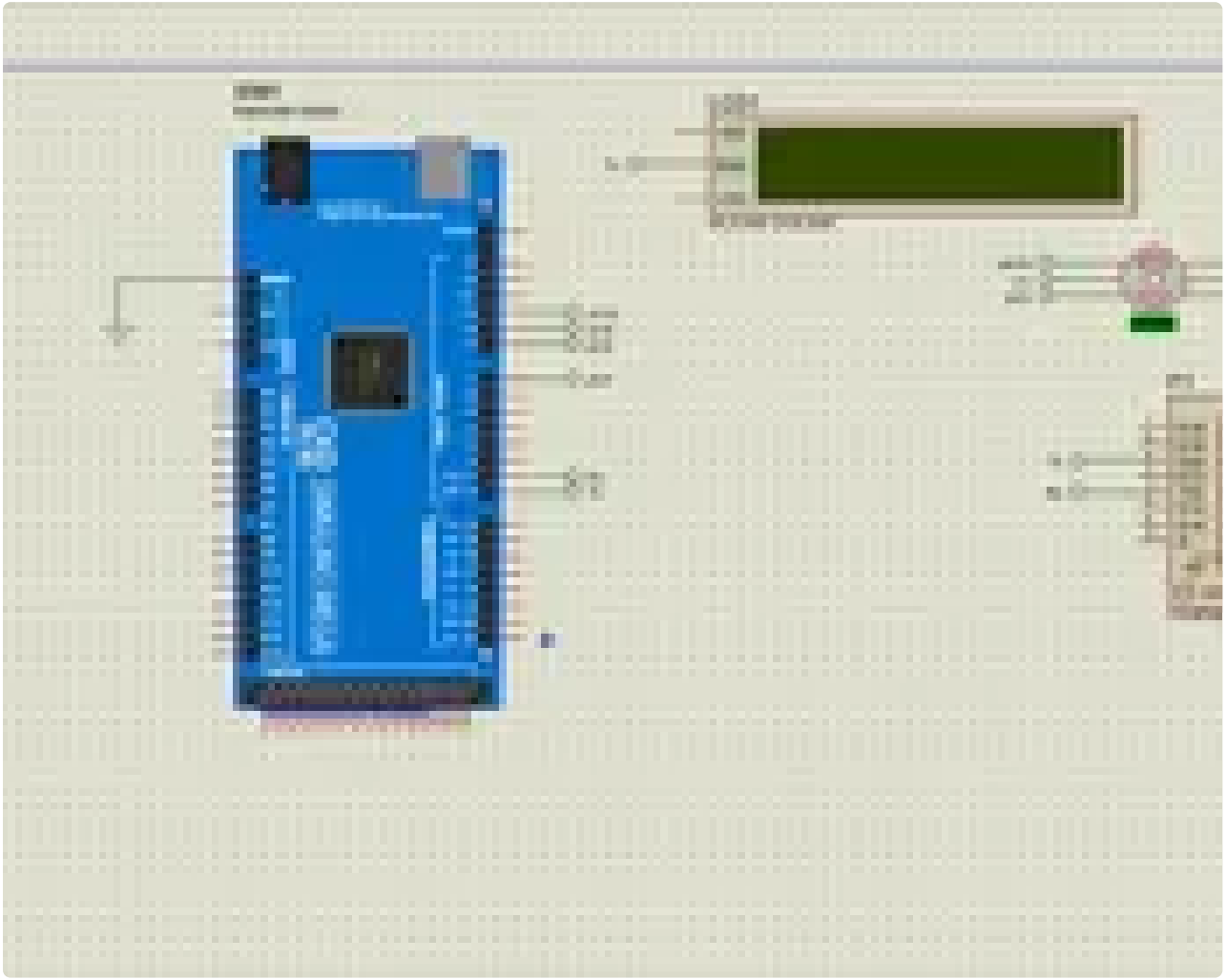
CONTROL STEPPER MOTOR SPEED WITH LABVIEW AND ARDUINO

By joony786 in Circuits > Arduino 32.217 141 12 Featured

(cc) BY-NC-SA



I Made this in Proteus

u CONTROL STEPPER MOTOR SPEED WITH LABVIEW AND ARDUINO...!! wolaa

Step 1: This Tutorial Is:

> Using Labview to interface Arduino to PC via USB port.

> Using Labview to send data that describes Motor Speed to Arduino without any toolkits.

> Solving the problem of Arduino serial port hanging.

> Using Arduino to generate Stepping pulses.

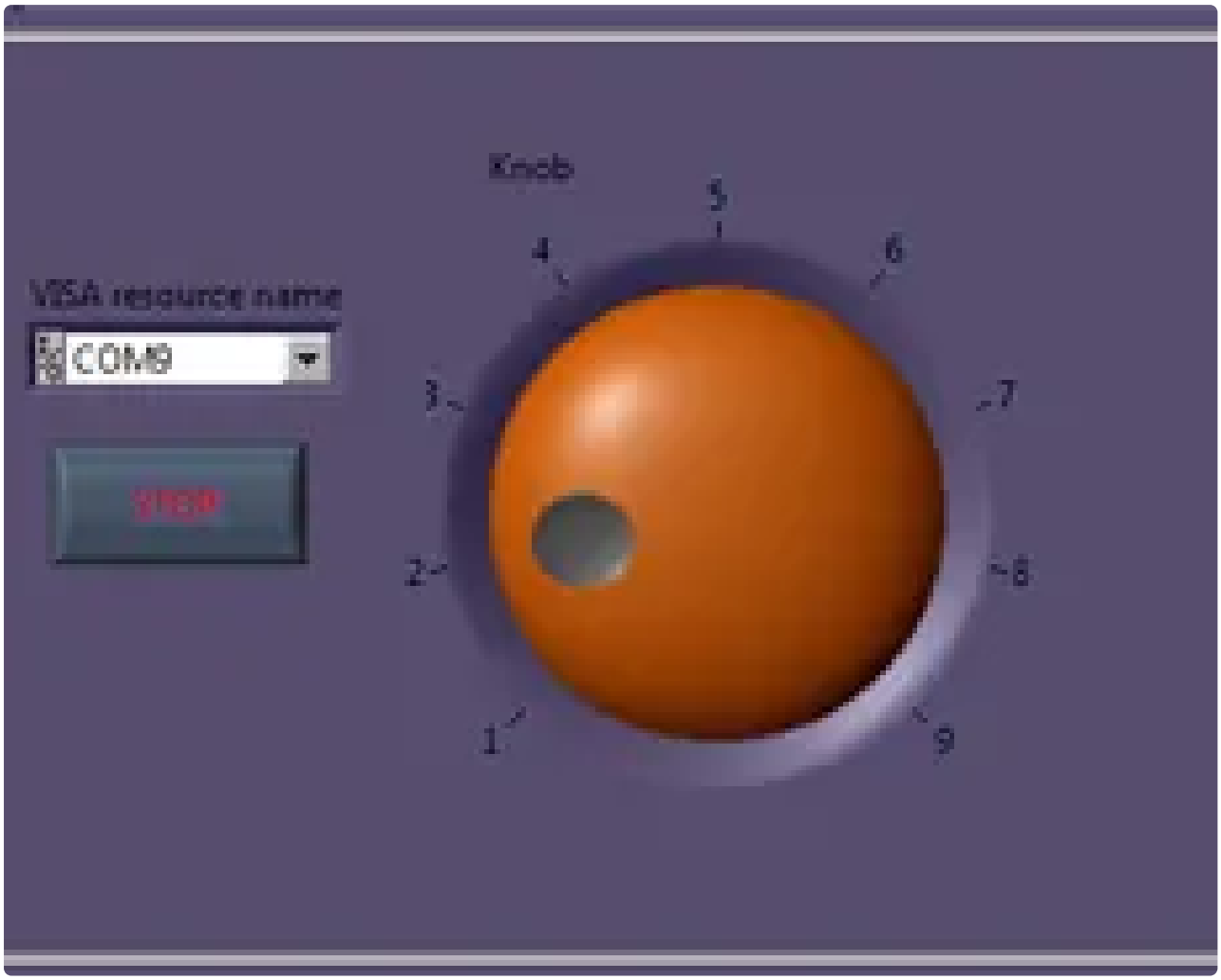
> Using Arduino Mega's Serial port 1 to show stepper motors speed on a serial LCD.

The best way to interface Arduino to PC is via its USB port. Arduino uses virtual Serial COM port to interface its USB connector to PC. At the same time Arduino uses its USB to interface its own serial (USART) port…. Bravo Arduino!!!

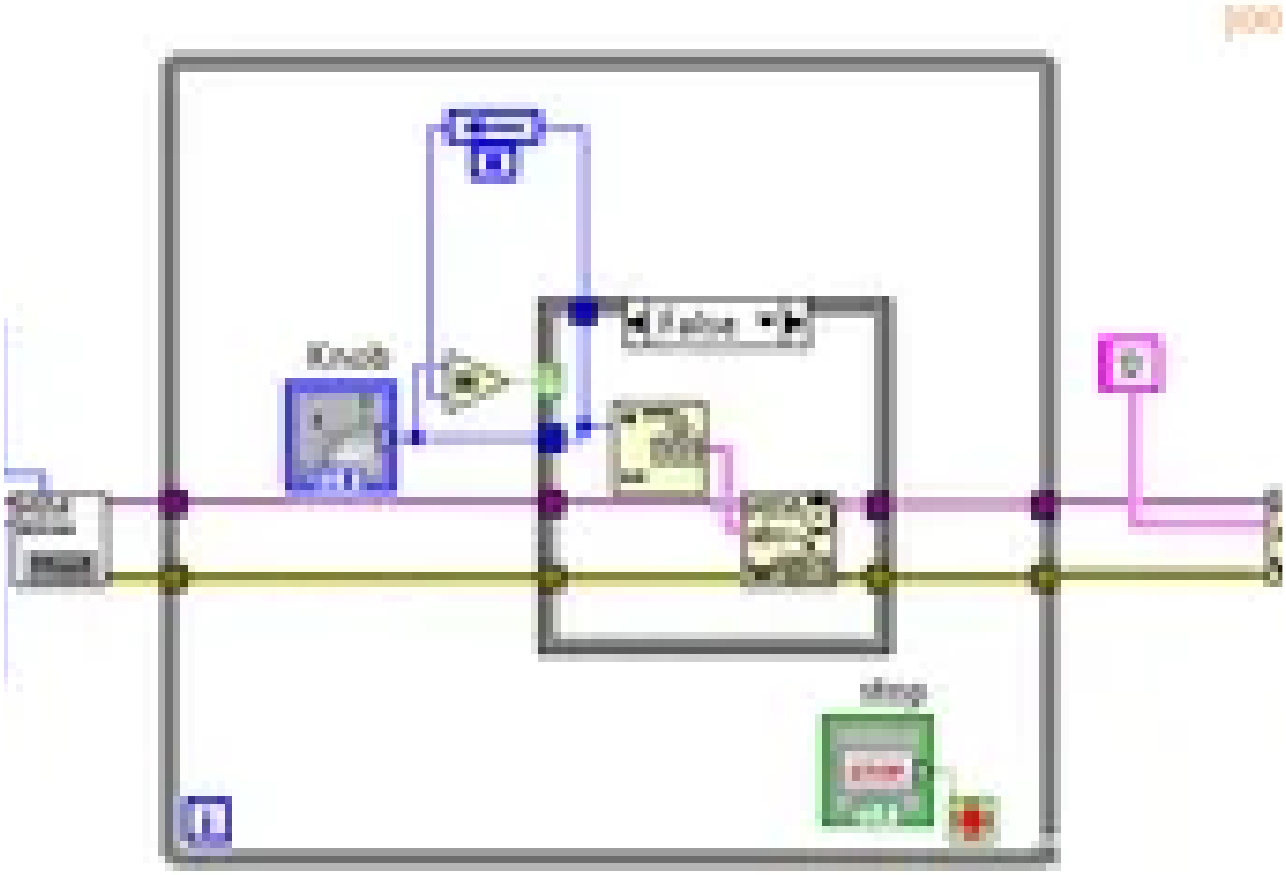
To send data to Arduino from your computer just connect it to USB port and use a program that can deal with computers serial port. This program can be Aeduino Serial port monitor, MIKRO C Serial terminal tool, PIC MP-LAB, or a program you establish. The one you build on yourself can be made by using a programming language like C, C#, C++, Python, Basic, NI-LABVIEW… etc.

Your own program not only has the advantage of interfacing the port, but performing a specific mission you need in your project as well. And so I do down here.

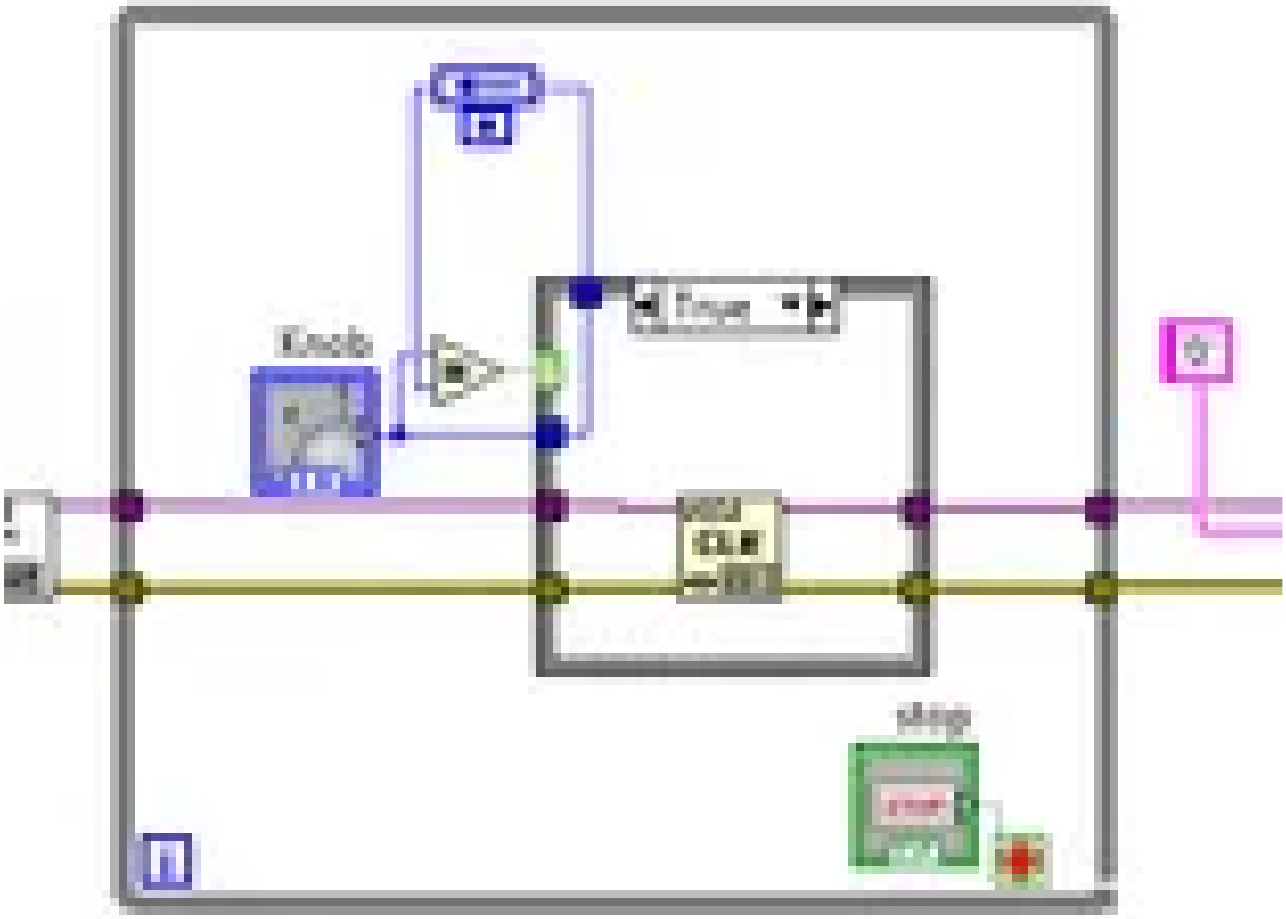
Step 2: In Labview (any Version) Establish the Following Front Panel



Step 3: Establish the Following Block Diagram. (click to Enlarge)



Step 4: And Here Is the Another Case of (if) Statement.



Step 5: This Virtual Instrument:

>> Starts communication with Arduino.

>> Detects if there is a change in control knob value.

>> If there is a change it sends the new value to Arduino.

>> Else, it clears serial port Input/output register to avoid data overflow and serial port overloading.

>> When pressing STOP, it sends “0” (brake or hold order) to Arduino.

Step 6: Arduino Code

Step 7: This Arduino Sketch:

Checks if labview sent a new value.If there is a new value, it converts it into integer byte with the same value.This integer byte represents the delay interval used in stepping sequence. That, the greater the delay the lower the speed and vice versa.Else, Arduino continue running the motor at current speed. This prevents the processor from overloading the serial port and pushes it to avoid Arduino Serial port hanging… Bravo ;)

Calculates motor speed from its resolution and delay intervals between steps as following:

Rpm = (60 x 1000 x step angle) / (360 x (delay m.sec))

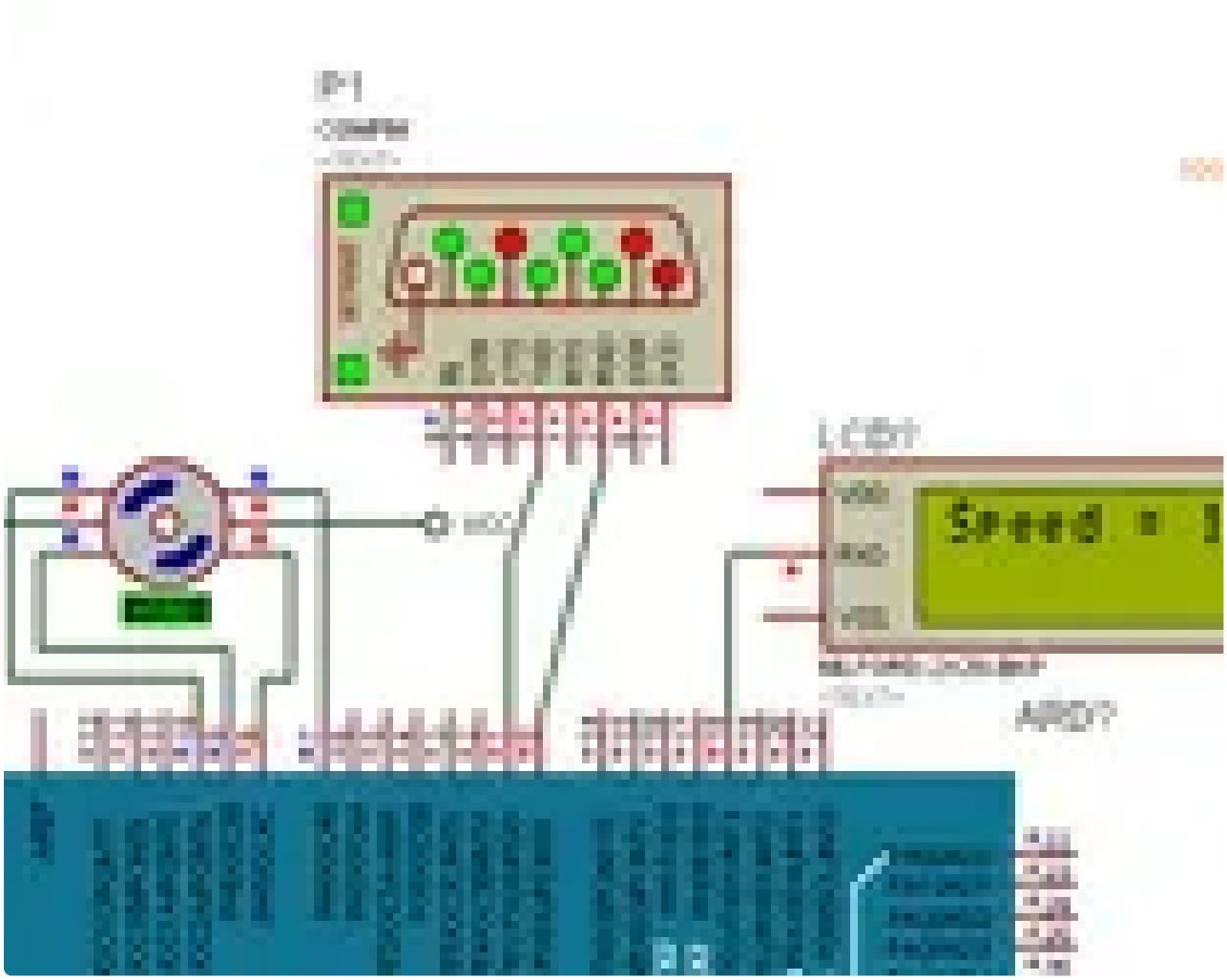
Rpm = (1000 x step angle) / (6 x (delay in msec))

I used a 1.8 deg/step motor in this project and so:

Rpm = 300 / (delay in msec)

Sends the calculated motor speed to another Serial port of Arduino Mega to show if our code works

Step 8:



I simulated this tutorial with PROTEUS and worked perfectly.

In real application, it controlled my stepper motor perfectly on the long run.

Step 9: Proteus File

Enjoy.. :)